



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده ریاضی و علوم کامپیوتر

گزارش تمرین درس داده کاوی محاسباتی
تمرین ۶

فشرده سازی مدل های عمیق با تجزیه رتبه پایین و ارتقای کیفیت صنعتی

نگارش
علی اصغر کرمی

استاد راهنما
دکتر مهدی قطعی

استاد مشاور
مهندس بهنام یوسفی مهر

دی ۱۴۰۴



به نام خدا

تاریخ: ۱۴۰۴/۱۰/۱۱

تعهدنامه اصالت اثر

اینجانب علی اصغر کرمی متعهد می‌شوم که مطالب مندرج در این پایان نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

امضا

چکیده

در این گزارش، تاثیر فشرده سازی لایه های یک شبکه عصبی عمیق به کمک تقریب های کم رتبه به طور عملی بررسی شده است و راه کار هایی نیز برای این امر مطرح شده است.

واژه های کلیدی:

تقریب کم رتبه، شبکه عصبی، یادگیری عمیق، فشرده سازی مدل عمیق

صفحه	فهرست مطالب
أ	چکیده.....
۱	فصل اول مقدمه.....
۲	مقدمه.....
۳	فصل دوم گزارش.....
۴	۲-۱- تحلیل طیفی وزن های مدل.....
۸	۲-۲- پیاده سازی عملی فشرده سازی.....
۱۰	۲-۳- بازیابی دقت و ارزیابی سرعت.....
۱۱	۲-۴- سوالات تحلیلی.....
۱۴	منابع و مراجع.....
۱۵	Abstract.....

صفحه

فهرست اشکال

شکل ۱- شبکه عصبی کانولوشنال.....	۴
شکل ۲- معماری مدل resnet.....	۵
شکل ۳- مدل resnet 18.....	۶
شکل ۴- نتیجه انتخاب برش های مختلف به کمک تجزیه مقادیر تکین روی وزن های لایه میانی انتخاب شده.	۷
شکل ۵- نتیجه انتخاب برش های مختلف به کمک تجزیه مقادیر تکین روی وزن های لایه تماما متصل انتهایی	۸
شکل ۶- نمودار تغییرات دقت بعد از فشرده سازی همراه با تنظیم دقیق.....	۱۱

صفحه

فهرست اشکال

جدول ۱- تعداد پارامتر ها و دقت مدل با و بدون فشرده سازی بر روی داده تست.....	۹
--	---

فصل اول

مقدمه

مقدمه

در این تمرین ابتدا مروری بر شبکه های عصبی کانولوشنال انجام می دهیم و بعد از آن به سراغ مدل resnet و ساختار آن میرویم سپس تلاش میکنیم به کمک یک تقریب کم رتبه مثل تقطیع با تجزیه مقادیر منفرد یا تکین این مدل را فشرده سازی کنیم و اثر آن را بر دقت بررسی کنیم، همچنین به تاثیر تنظیم دقیق در این موضوع میپردازیم و در انتها نیز راه کارهایی برای انتخاب لایه برای فشرده سازی و موارد این چنینی میپردازیم.

کد مربوط به این تمرین از بخش زیر قابل مشاهده میباشد:

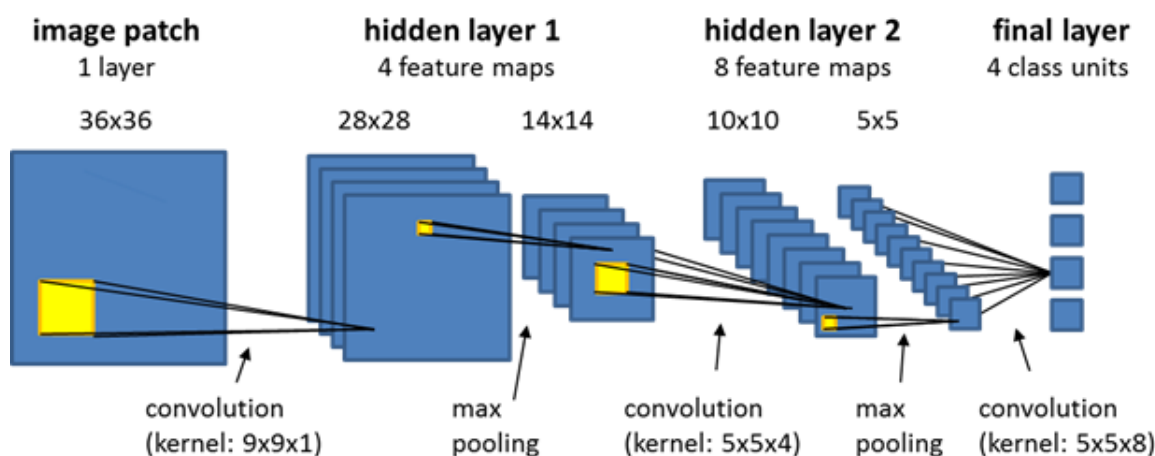
https://github.com/aliasghar892/datamining_SvdOnDeeplearning

فصل دوم

گزارش

۲-۱- تحلیل طیفی وزن های مدل

در ابتدا نیاز داریم با شبکه های عصبی کانولوشنال^۱ آشنا شویم. شبکه های عصبی کانولوشنال نوعی از مدل های یادگیری عمیق هستند که به طور خاص برای پردازش داده های دارای ساختار شبکه ای، مانند تصاویر، طراحی شده اند. این شبکه ها با الهام از سیستم بینایی انسان، به جای بررسی تک تک پیکسل ها به صورت جداگانه، از فیلتر هایی (کرنل ها) استفاده می کنند که روی تصویر حرکت کرده و ویژگی های مهم آن (مانند لبه ها، بافت ها و اشکال پیچیده) را به صورت خودکار استخراج می کنند؛ این ویژگی باعث می شود این مدل ها در تشخیص الگوهای تصویری بسیار سریع و دقیق عمل کنند و حجم محاسبات را نسبت به شبکه های عصبی معمولی به شدت کاهش دهند. مباحثی مانند max pooling و ... نیز وجود دارند که در اینجا به آنها نمی پردازیم. [1]



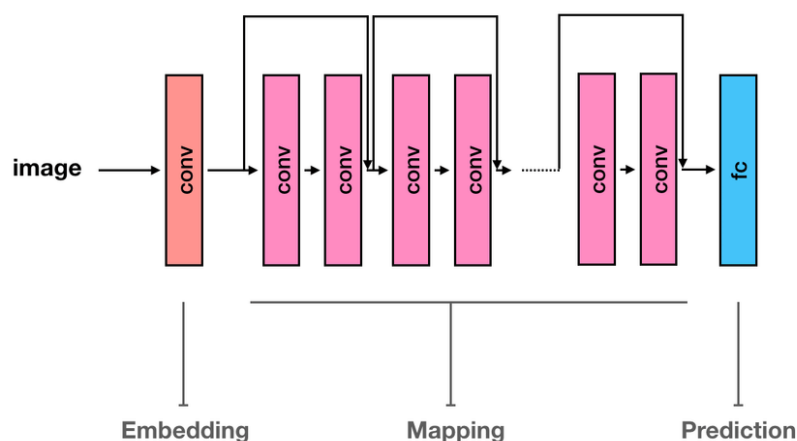
شکل ۱- شبکه عصبی کانولوشنال

حال نیاز داریم مدل resnet 18 را معرفی کنیم. شبکه های عصبی resnet یک معماری انقلابی در یادگیری عمیق هستند که با معرفی ایده اتصالات میانبر^۲، مشکل آموزش شبکه های بسیار عمیق را حل کردند. در این روش، به جای اینکه لایه ها صرفاً به دنبال یادگیری نگاشت اصلی باشند، تلاش می کنند تا تفاوت یا باقی مانده بین ورودی و خروجی مطلوب را یاد بگیرند و ورودی لایه قبلی را مستقیماً به

^۱ Convolutional neural network

^۲ Skip connections

خروجی لایه فعلی اضافه می‌کنند. این کار مانع از مشکل محو شدن گرادیان^۳ شده و اجازه می‌دهد شبکه‌هایی با صدها لایه با دقت بسیار بالا و پایداری مناسب آموزش ببینند. [2]



شکل ۲- معماری مدل resnet

resnet-18 یک نسخه سبک و کارآمد از خانواده شبکه‌های عصبی resnet است که دقیقاً از ۱۸ لایه یادگیری (شامل لایه‌های کانولوشن و تمام‌متصل) تشکیل شده و برای ایجاد تعادل میان سرعت پردازش و دقت طراحی شده است. این مدل برخلاف نسخه‌های عمیق‌تر که از ساختار پیچیده‌تر گلوگاهی^۴ استفاده می‌کنند، بر پایه بلوک‌های ساده^۵ بنا شده که هر بلوک تنها شامل دو لایه کانولوشنال 3×3 است. این ساختار ساده در کنار اتصالات میانبر به شبکه اجازه می‌دهد تا بدون گرفتار شدن در مشکل محو شدن گرادیان، ویژگی‌های بصری را به‌خوبی یاد بگیرد و به همین دلیل به عنوان یکی از محبوب‌ترین مدل‌های پایه برای شروع پروژه‌های بینایی ماشین و کاربردهای سریع شناخته می‌شود. [2]

حال قصد داریم ساختار این شبکه را به صورت دقیق‌تر بررسی کنیم. این شبکه ابتدا از لایه ورودی یا ساقه^۶ شروع میکند که در آن یک پردازش بر روی تصویر ورودی به کمک یک شبکه کانولوشنال رخ

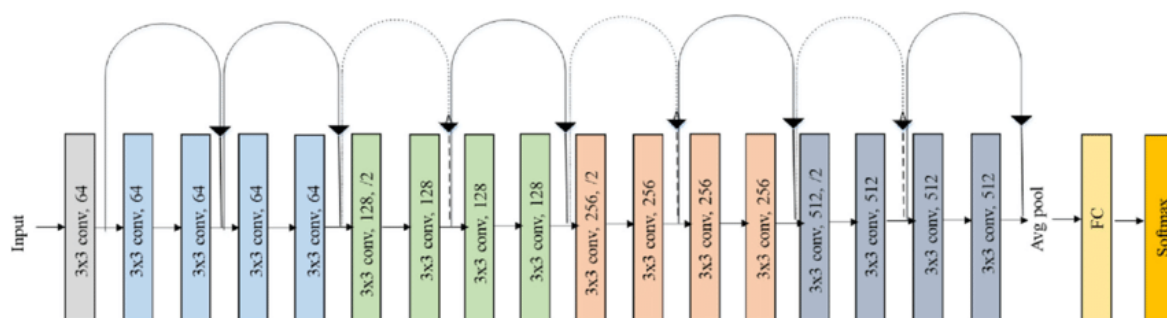
^۳ Vanishing gradient

^۴ bottleneck

^۵ Basic blocks

^۶ stem

میدهد، بعد از آن ۴ مرحله^۷ وجود دارد که هر کدام از آنها شامل دو بلوک پایه میباشند، هر بلوک پایه نیز شامل دو لایه کانولوشنال و اتصال های میانبر میباشد. در نهایت نیز یک لایه تماما متصل داریم که طبقه بندی را انجام میدهد. [2]



شکل ۳- مدل resnet 18

حال در ابتدای این تمرین مدل resnet-18 از قبل آموزش دیده^۸ را بارگذاری میکنیم، لایه کانولوشنال اول بلوک اول در مرحله دوم را به عنوان یک لایه میانی و لایه تماما متصل انتهایی را انتخاب میکنیم.

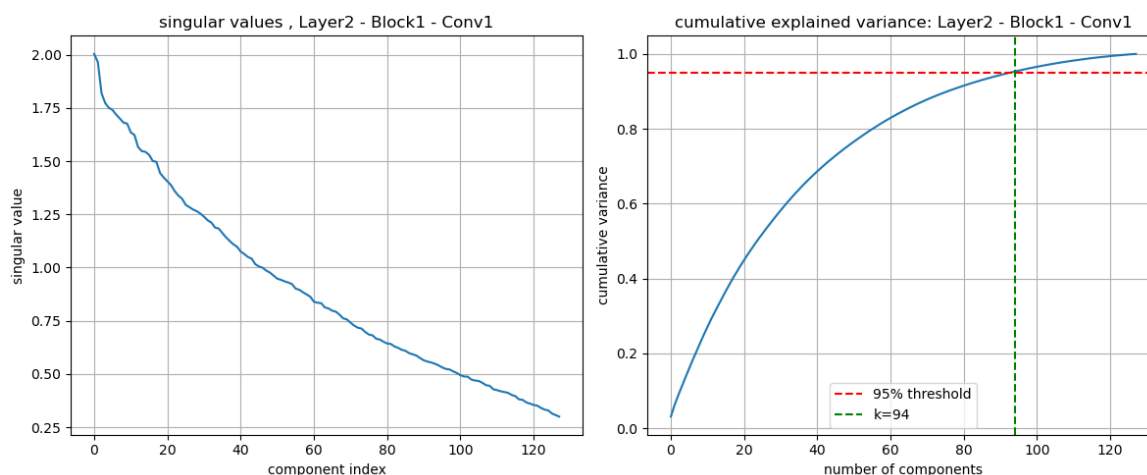
وزن های لایه میانی انتخاب شده دارای بعد ۱۲۸ در ۶۴ در ۳ در ۳ میباشند (۱۲۸ تعداد کرنل ها، ۶۴ عمق هر کدام و ۳ در ۳ طول و عرض آنها میباشد). برای استفاده موثر از تجزیه مقادیر تکین به این شکل عمل میکنیم که ابتدا هر فیلتر یا کرنل سه بعدی را مسطح^۹ یا به عبارتی دیگری به یک بردار تبدیل میکنیم. حال یک ماتریس جدید میسازیم که هر سطح آن یک فیلتر مسطح و ستون های ساختار فیلتر را نشان میدهند (ابعاد این ماتریس ۱۲۸ در ۵۷۶ خواهد بود). حال میتوانیم بررسی کنیم که آیا به وجود همه این فیلتر ها نیاز داریم یا میان آنها افزونگی وجود دارد. برای این امر از تجزیه مقادیر تکین استفاده میکنیم.

^۷ stage

^۸ pretrained

^۹ flatten

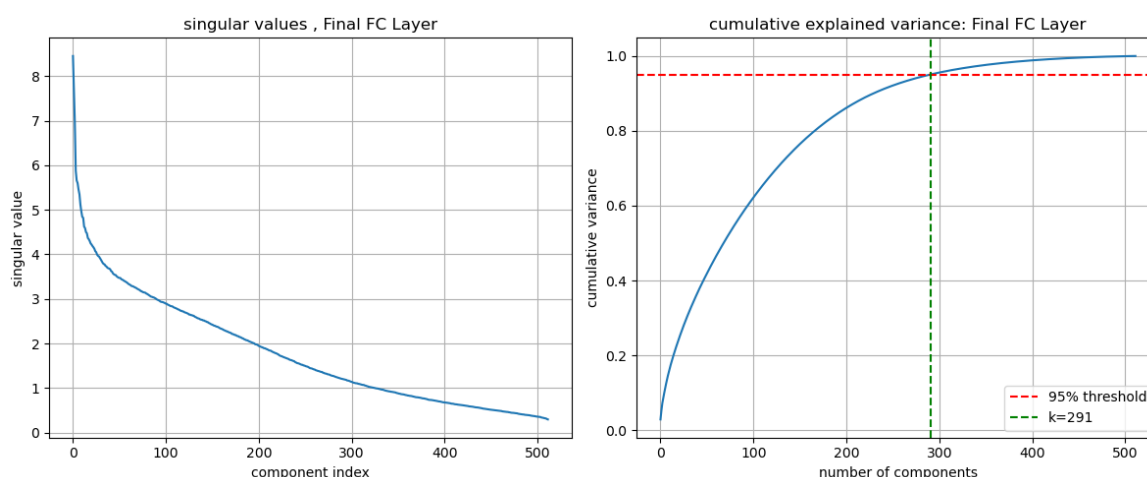
از آنجایی که مربع مقادیر تکین، مقادیر ویژه را نشان می‌دهند، می‌توانیم با جمع مربع همه آنها واریانس کل را محاسبه کنیم و ببینیم با نگه داشتن هر مقدار از این مقادیر تکین، چه مقدار از واریانس توضیح داده می‌شود.



شکل ۴- نتیجه انتخاب برش‌های مختلف به کمک تجزیه مقادیر تکین روی وزن‌های لایه میانی انتخاب شده

در سمت چپ شکل ۴، مقادیر تکین را در قالب یک نمودار مشاهده می‌کنیم، روند کاهش مقدار تکین برای مقادیر اول بسیار شدید و بعد از آن یک روند نزولی نسبتاً خطی وجود دارد. در نمودار سمت راست نیز روند افزایش واریانس توضیح داده شده بعد از اضافه کردن مقادیر تکین را مشاهده می‌کنید، همچنان در تصویر مقدار ۹۵ درصد واریانس توضیح داده شده مشخص شده است و بعد از آستانه گذاری روی آن ۹۴ مقدار تکین از بین ۱۲۸ (تعداد کل مقادیر تکین) انتخاب می‌شوند. (به عبارتی ۷۳.۴۴ درصد آنها انتخاب می‌شوند)

لایه تماماً متصل انتهایی از ابتدا ساختار ماتریسی با اندازه ۱۰۰۰ در ۵۱۲ دارد و مجدداً روی آن می‌توانیم تجزیه مقادیر تکین اعمال کنیم.



شکل ۵- نتیجه انتخاب برش های مختلف به کمک تجزیه مقادیر تکین روی وزن های لایه تماما متصل انتهایی

مجدداً از نمودار سمت چپ شکل ۵ مشخص است که روند نزول شدید تری در مقادیر تکین وجود دارد و احتمالاً برای حفظ ۹۵ درصد واریانس توضیح داده شده باید برش بیشتری بزنیم (تعداد ویژگی های بیشتری دارای افزونگی هستند). در سمت راست نیز میبینیم که برای این برش کفایت از بین ۵۱۲ مقادیر تکین، ۲۹۱ از آنها را انتخاب کنیم و این یعنی فقط ۵۶.۸۴ درصد از آنها.

۲-۲- پیاده سازی عملی فشرده سازی

در این بخش قصد داریم مستقیماً مدل resnet-18 را فشرده سازی کنیم و نتایج را بر روی داده cifar 10 بررسی کنیم.

دو نکته مهم در این مرحله وجود دارد:

اول اینکه لایه تماماً متصل تنها لایه خطی مدل resnet میباشد [2]. از آنجایی که خواسته سوال فشرده کردن لایه خطی است پس فقط همین لایه را فشرده میکنیم.

دوم، داده cifar 10، فقط ۱۰ کلاس دارد. نیاز داریم لایه تماماً متصل مدل را با یک لایه خطی دیگر که ده خروجی دارد جایگزین کنیم. این موضوع باعث از بین رفتن وزن های یادگرفته شده میشود و دقت مدل شدیداً کاهش پیدا میکند. در چنین موقعیتی نیاز به تنظیم دقیق^{۱۰} وجود دارد، به همین منظور

^{۱۰} Fine tuning

قبل از فشردن سازی برای epoch ۵ تنظیم دقیق بر روی داده جدید انجام می‌دهیم تا مقایسه انجام شده اصولی باشد و بر اساس وزن های تصادفی نباشد.

برای فشردن سازی به این صورت عمل می‌کنیم که لایه تماما متصل را به عنوان ورودی گرفته، روی آن تجزیه مقادیر تکین اعمال می‌کنیم، سپس برای تقریب کم رتبه از روش تقطیع تجزیه مقادیر تکین^{۱۱} استفاده می‌کنیم. به طور دقیق تر نیاز داریم ماتریس وزن را به شکل زیر بازسازی کنیم (مقدار k را بر اساس مقدار فشردن سازی که قصد داریم انجام دهیم تعیین می‌کنیم، کلیات این روش در گزارش ۱ توضیح داده شده بود) [3]:

$$W_{approx} = U_k \Sigma_k V_k^t$$

برای این امر ماتریس را به دو بخش تقسیم می‌کنیم:

$$W_2 = U_k \Sigma_k^{\frac{1}{2}} \quad W_1 = \Sigma_k^{\frac{1}{2}} V_k^t$$

با ضرب این دو همان ماتریس وزن های تقطیع شده به دست می آید، حال اگر لایه تمام متصل را به دولایه خطی به ترتیب با وزن های W_1 و W_2 تبدیل کنیم، لایه اول ورودی را طبق روال قبل می‌گیرد و به لایه دوم می‌دهد و لایه دوم نیز پیش بینی را انجام می‌دهد. با این کار لایه تماما متصل خطی در دو لایه فشردن خلاصه می‌شود (به کمک تجزیه مقادیر تکین سعی شده که ساختار وزن ها همزمان با فشردن سازی حفظ شوند) که عملکرد آن را در جدول زیر بررسی می‌کنیم.

جدول ۱- تعداد پارامتر ها و دقت مدل با و بدون فشردن سازی بر روی داده تست

توضیحات	تعداد پارامتر	دقت روی داده تست
مدل بدون فشردن سازی	11,181,642	87.11%
مدل با فشردن سازی ۵۰ درصد	11,179,132	76.95%
مدل با فشردن سازی ۸۰ درصد	11,177,566	38.87%

^{۱۱} Truncated svd

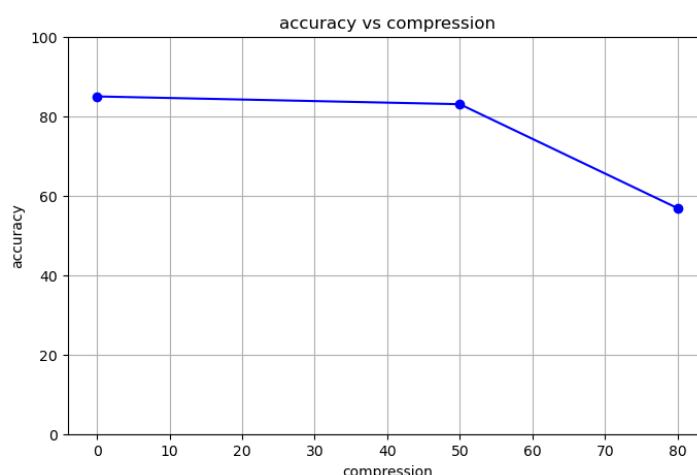
همان طور که مشخص است تعداد قابل توجهی از پارامترهای مدل کاهش پیدا کرده است (اگر چه به نسبت کل پارامترها این تغییر کمتر از ۱ درصد است، برای فشرده سازی بیشتر میتوان به روش مرحله ۱ و با اندکی تغییر لایه های کانولوشنال را نیز فشرده کرد) و دقت مدل بعد از ۵۰ درصد فشرده سازی هنوز مطلوب است و تغییر زیادی نکرده، ولی با فشرده سازی ۸۰ درصد دقت افت زیادی داشته. در مرحله بعد خواهیم دید با تنظیم دقیق بعد از فشرده سازی، میتوان وزن ها را به کمک داده آموزشی به روزرسانی کرد و دقت مدل را با توجه به ساختار جدید بهبود داد.

۲-۳- بازیابی دقت و ارزیابی سرعت

در این مرحله قصد داریم بررسی کنیم که بعد از تنظیم دقیق بعد از فشرده سازی آیا دقت مدل افزایش پیدا میکند یا خیر.

ابتدا مدل ۵۰ درصد فشرده را به اندازه ۳ ایپاک تنظیم میکنیم، بعد از این امر دقت این مدل از ۷۲.۸۵ درصد به ۸۳.۰۱ افزایش میابد، که بسیار به دقت مدل بدون فشرده سازی نزدیک میباشد. این امر نشان میدهد که نه تنها تعداد پارامترهای لایه تماما متصل را ۵۰ درصد کاهش دادیم، بلکه تقریباً عملکرد مدل نیز حفظ شده و دقت خوبی از خود در داده های تست نشان میدهد.

در مدل ۸۰ درصد فشرده نیز دقت از ۴۰.۴۳ به ۵۶.۸۴ افزایش میابد، این امر نیز نشان دهنده اهمیت تنظیم دقیق بعد از فشرده سازی میباشد و احتمالاً (بر اساس رفتار خطا در طول این سه ایپاک) بعد از ایپاک های بیشتر دقت بیشتر به مدل بدون فشرده سازی نزدیک شود. لازم به یادآوری است که لایه تماماً متصل این مدل ۸۰ درصد فشرده شده است و این دقت برای این مدل ارزشمند میباشد.



شکل ۶- نمودار تغییرات دقت بعد از فشرده سازی همراه با تنظیم دقیق

در نهایت نیز سرعت استنتاج را بر روی مدل اصلی و مدل های فشرده بررسی میکنیم (سرعت بر روی cpu اندازه گیری شده است). لازم به ذکر است برای این امر یک داده ساختگی ۱۰۰۰ بار به عنوان ورودی به مدل داده شده و زمان میانگین گزارش شده است. علت این امر نوسان بالای زمان استنتاج میباشد (کوچک بودن مقیاس مقایسه یعنی میلی ثانیه نیز باعث افزایش نوسان میشود). با این وجود نتایجی که در زیر گزارش شده اند همچنان نوسان دارند و حتی ممکن است بعد از تکرار مجدد آزمایش سرعت مدل فشرده کمتر از مدل اصلی باشد!

در نهایت سرعت استنتاج بر روی مدل اصلی به طور میانگین ۵۳.۸۰ میلی ثانیه، برای مدل ۵۰ درصد فشرده ۴۰.۹۷ ثانیه (۱.۳۱ برابر سریع تر) و برای مدل ۸۰ درصد فشرده ۴۸.۸۸ ثانیه (۱.۱ برابر سریع تر) میباشد.

سرعت استنتاج بر روی gpu نیز برای مدل اصلی ۲.۴۷ میلی ثانیه میباشد و تغییر سرعت مدل های فشرده ناچیز میباشد.

۲-۴- سوالات تحلیلی

۱. تاثیر تأخیر ناشی از افزایش تعداد لایه ها را در محیط های مختلف تحلیل کنید.

به طور کلی با افزایش تعداد لایه ها، تعداد پارامتر ها افزایش میابد و بدیهی است که محاسبات مورد نیاز برای آموزش و استنتاج به شدت افزایش میابند.

ولی در شرایطی مثل شرایط بالا که همزمان با افزایش تعداد لایه ها، تعداد پارامتر ها را کاهش دادیم، شرایط مقداری فرق میکند. از نظر آزمایشی به دلیل محدودیت های اندازه گیری در چنین مقیاسی تنها چیزی که میتوان گفت این است که در محیط دارای gpu سرعت محاسبات بسیار بیشتر میباشد و ممکن است این افزایش سرعت به حدی باشد که حتی تاخیر ناشی از افزایش لایه ها تا حدی حتی قابل اندازه گیری نیز نباشد. از لحاظ تئوری، وقتی یک لایه را به دولایه میشکنیم، لایه دوم نمیتواند قبل از اتمام محاسبات لایه اول شروع به محاسبات کند و این موضوع اثر موازی سازی را کم میکند و ممکن است باعث تاخیر با وجود کم شدن تعداد پارامتر ها شود.

در مورد هم cpu و هم gpu، درست است که با شکاندن یک لایه بزرگ به دو لایه کوچک تر به روشی که انجام دادیم، تعداد پارامتر ها و محاسبات کاهش میابد ولی همچنان گلوگاه هایی مثل حافظه وجود دارند، برای مثال در حالت تک لایه، یک لایه وارد حافظه میشود و محاسبات روی آن انجام و خارج میشود، ولی در حالت دو لایه این امر اتفاق نمی افتد و پس از خروج لایه اول، لایه دوم باید وارد شود و این موضوع میتواند بسته به حافظه باعث تاخیر شود. (این تاخیر نیز بسته به قدرت حافظه میتواند ناچیز باشد)

۲. چگونه می توان از مفاهیم «عدد وضعیت» برای انتخاب لایه های مستعد فشرده سازی استفاده کرد؟

وقتی عدد وضعیت بزرگ است (اختلاف بین بزرگترین و کوچکترین مقدار منفرد بسیار زیاد است)، یعنی ساختار و اطلاعات ماتریس فقط در چند مؤلفه اول متمرکز شده است و بقیه مؤلفه ها مقادیر ناچیزی دارند (نویز یا اطلاعات کم اهمیت)، در نتیجه ما می توانیم با خیال راحت بخش زیادی از مقادیر کوچک را حذف یا تقطیع کنیم بدون اینکه اطلاعات مهمی از دست برود. این لایه ها بهترین کاندید برای فشرده سازی هستند. (زیرا با کمترین از دست رفتن ساختار، بیشترین فشرده سازی را میدهند)

در مقابل وقتی عدد وضعیت کوچک است، یعنی همه مقادیر منفرد تقریباً هم اندازه هستند. در نتیجه تمام ابعاد ماتریس حاوی اطلاعات مهم هستند و حذف کردن هر کدام از آنها باعث افت شدید دقت مدل می شود. بنابراین، این لایه ها را نباید فشرده کرد (یا باید با احتیاط خیلی زیاد فشرده شوند).

بنابراین بهتر است ابتدا لایه های با عدد حالت بالا را به ترتیب فشرده سازی کنیم و سپس سراغ لایه های با عدد حالت پایین برویم.

برای مثال به شکل ۴ و ۵ مراجعه کنید، از شکل ۵ مشخص است که تعداد اندکی از مقادیر تکین دارای مقدار بالا و بقیه دارای مقدار کمی هستند (در شکل ۴ مقدار کاهش مقادیر تکین کمتر است و دیر تر اتفاق میافتد)، بنابراین توقع داریم که عدد حالت لایه انتهایی بیشتر از لایه میانی انتخاب شده باشد. بعد از محاسبه دقیق عدد حالت این دو لایه، عدد حالت لایه میانی ۶.۶۸ و عدد حالت لایه انتهایی ۲۵.۵۱ می باشد و این موضوع مهر تاییدیست بر توضیحات داده شده.

۲-۵- مطالعات بیشتر

فشرده سازی مدل های عمیق بدون از دست دادن دقت موضوع بسیار با اهمیتی می باشد، مهم ترین دلیل آن این است که سخت افزار ها و gpu های قدرتمند همیشه در اختیار نیستند و گاهی نیاز داریم مدل را برای اجرا پذیر بودن بر روی دستگاه های سبک تر مثل تلفن های همراه و ... فشرده کنیم. در نهایت لازم به ذکر است که مطالعه انجام شده در این گزارش صرفاً در حد یک تمرین نمی باشد و تحقیقات جدی روی آن صورت گرفته است که میتوان به آنها مراجعه کرد. [4,5,6,7]

منابع و مراجع

- [1] Zewen Li, Wenjie Yang, Shouheng Peng, Fan Liu, Member. 2020. A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. 2015. Deep Residual Learning for Image Recognition.
- [3] Zhihua Zhang. 2015. The Singular Value Decomposition, Applications and Beyond.
- [4] Remi Denton, Wojciech Zaremba, Joan Bruna, Yann LeCun, Rob Fergus. 2014. Exploiting Linear Structure Within Convolutional Networks for Efficient Evaluation.
- [5] L. Liebenwein, A. Maalouf, O. Gal, D. Feldman. 2021. Compressing Neural Networks: Towards Determining the Optimal Layer-wise Decomposition.
- [6] Y.D. Kim, E. Park, S. Yoo, T. Choi. 2016. Compression of Deep Convolutional Neural Networks for Fast and Low Power Mobile Applications
- [7] Max Jaderberg, Andrea Vedaldi, Andrew Zisserman. 2014. Speeding up Convolutional Neural Networks with Low Rank Expansions

Abstract

In this report, the effect of compressing the layers of a deep neural network using low-rank approximations is practically investigated, and solutions for this are also proposed.

Key Words: Low-rank approximation, neural network, deep learning, deep model compression



**Amirkabir University of Technology
(Tehran Polytechnic)**

Department of Mathemaics and Computer Science

report on sixth exercise of Computatinal data mining

Deep model compression with low-rank decomposition

**By
Aliasghar karami**

**Supervisor
Dr. Mahdi Ghati**

**Advisor
Mr. Behnam Yousefi Mehr**

December , 2025